PART 1

System overview and objective

Paving Service Request and Job Management System

I plan to develop a Paving Service Request and Job Management System using ASP.NET Core MVC. The system will give customers a convenient way to request paving work and give the business a central place to manage requests and jobs. It will organize information that might otherwise be spread across phone calls, emails, and handwritten notes.

Target users and access

The system will serve visitors, registered customers, employees, and administrators. Visitors who are not logged in will be able to browse available services and submit a basic service request. Registered customers will be able to sign in and view the status of their own requests. Employees will see assigned jobs and update progress and notes. Administrators will review requests, schedule jobs, assign employees, and manage customer and user records.

Data collection and management

The completed system will use a relational database through Entity Framework Core, with SQL Server LocalDB planned for course development. It will collect names, email addresses, phone numbers, property addresses, service selections, and project descriptions. Job records will also contain dates, estimated costs, statuses, employee assignments, and notes. These fields include text, numeric identifiers, decimal currency amounts, and date/time values.

Customers and staff will create records through forms, read information through dashboards and detail pages, and update permitted information as work progresses. Administrators will be able to remove eligible duplicate or incorrect records, with confirmation and relationship checks. Records linked to active jobs will be retained or archived instead of being deleted. Access to stored information will depend on the user's role and relationship to the record.

Public web service

The system will integrate the Open-Meteo Forecast API to display temperature and precipitation forecasts for a job's location. Staff will use a city or postal-code lookup to select coordinates for the forecast. The application will request weather data over HTTPS, read the JSON response, and display the location, forecast date, units, and retrieval time. If the service is unavailable, the application will show a clear notice while keeping job management available.

Objective and value

The main objective is to make service requests easier to submit and job information easier to manage. A shared record of each job will help reduce missing information, clarify responsibilities, and improve communication with customers. Part II establishes the project shell, requirements, and interface prototypes. Database persistence, account access, and live weather integration are planned for later course sections.

API reference: open-meteo.com/en/docs

PART 2

Project shell and initial development

Visual Studio solution

The project is named **PavingServiceManagement**. It uses the ASP.NET Core Web App (Model-View-Controller) template and targets **.NET 8**. The included .sln file opens in Visual Studio 2022. This target matches the development tools installed on the computer; the assignment's older .NET Core 3.1 tutorial provides the architectural reference.

Implemented shell

The landing page introduces the system and links to the service catalog, request-form prototype, dashboard preview, and project plan. Shared navigation and responsive styling are defined in a common layout. The pages use controllers, typed C# models, and Razor views rather than separate static HTML files.

MVC component	Responsibility	Examples
Models	Describe display and form data; define validation rules.	ServiceOption, ServiceRequestInput, JobPreview
Views	Render the user interface and validation feedback.	Home/Index, Requests/Create, Dashboard/Index
Controllers	Handle routes, supply model data, and validate submitted input.	HomeController, RequestsController, DashboardController
Services	Supply the read-only sample service catalog.	ServiceCatalog

The request form demonstrates model binding, required fields, email and phone validation, allowed service selections, and anti-forgery protection. A valid submission displays an explicit prototype message; no request is saved or sent. The dashboard uses fictional records and calculates its summary counts from those records.

Database storage, sign-in, role enforcement, job editing, and live API calls are planned, not implemented. The dashboard is a public design preview.

Run and review

Open **PavingServiceManagement.sln**, choose the **http** launch profile, and press **Ctrl+F5**. The local site runs at **http://localhost:5034**. The README includes a code tour and review steps. Use fictional information when testing the form.

Version control and submission

The project includes a .gitignore for Visual Studio settings, build output, credentials, and future database files. Source files and assignment documents belong in the GitHub repository. The repository URL and publication status are recorded in **REPOSITORY.md**. The submission guide identifies the files to upload and the link to provide to the instructor.

References: Microsoft ASP.NET Core MVC tutorial; Microsoft .NET support policy.

Functional requirements

The following requirements describe the intended completed system, not features already implemented in the Part II shell.

FR-01. As a visitor, I want to browse paving services without signing in so that I can decide which service fits my project.
Acceptance: Service descriptions remain publicly accessible.

FR-02. As a visitor, I want to submit contact details, a property address, a service type, and project notes so that the company can review my request.
Acceptance: Valid input creates one request with a reference number; invalid input is not saved.

FR-03. As a customer, I want to register, sign in, and sign out so that I can access my personal service information.
Acceptance: New accounts receive only the Customer role; failed login attempts return a generic error.

FR-04. As a customer, I want to view my requests and job statuses so that I can follow progress.
Acceptance: A customer cannot retrieve another customer's record, even by changing its URL.

FR-05. As a customer, I want to edit or cancel my own unreviewed request so that I can correct information or change my plans.
Acceptance: Reviewed requests require staff assistance; the original record remains traceable.

FR-06. As an employee, I want to view my assigned jobs so that I know where and when to work.
Acceptance: Employees see only their assignments and cannot manage user roles.

FR-07. As an employee, I want to update assigned-job progress and notes so that the office has current information.
Acceptance: Changes record the employee and time; internal notes stay hidden from customers.

FR-08. As an administrator, I want to review requests, create jobs, set estimates and dates, and assign employees so that work can be organized.
Acceptance: Saved changes appear after reloading the relevant record.

FR-09. As an administrator, I want to search and filter requests by customer, service, and status so that I can find work quickly.
Acceptance: Filters return matching records and show a clear empty state.

FR-10. As an administrator, I want to maintain customers and accounts and remove eligible incorrect records so that data stays accurate.
Acceptance: Role changes are restricted; deletion requires confirmation and cannot orphan jobs.

FR-11. As an employee or administrator, I want a forecast for the job location so that I can consider weather while scheduling.
Acceptance: The display identifies the location and time; unavailable or stale data is labeled.

Quality, access, and data requirements

NFR-01 / Performance. Under a course test load of 20 simultaneous users and 1,000 requests, at least 95% of ordinary page responses should complete within two seconds, excluding external API waits.

NFR-02 / Security. Use ASP.NET Core Identity for password hashing and sign-in. Enforce roles and record ownership on the server; use HTTPS when deployed and anti-forgery checks on state-changing forms.

NFR-03 / Validation. Reject missing required fields, invalid email/service values, negative estimates, and invalid dates. Use server-side validation and parameterized database access through EF Core.

NFR-04 / Usability. Provide labeled fields, visible keyboard focus, clear errors, readable contrast, and responsive layouts from 360px to desktop widths. Status must be understandable without color alone.

NFR-05 / Reliability. Save related database changes atomically. Time out weather requests after five seconds, cache results for 15 minutes, and keep request/job features usable if the API fails.

NFR-06 / Privacy. Limit access to contact details and property addresses; keep credentials and real customer data out of GitHub and logs. Guest requests are linked to an account only after ownership is verified.

NFR-07 / Maintainability. Separate models, views, controllers, and external-service logic. Use meaningful names, Git commits, a setup README, and reproducible database migrations.

NFR-08 / Recovery and compatibility. Back up the database before schema changes and verify restoration. Test core flows in current Edge and Chrome. The completed system must retain saved records after a restart.

Planned access levels

User type	Permitted access
Visitor (not signed in)	Browse services; create a basic request; no access to stored request lists.
Customer	View own requests/status; edit or cancel own unreviewed request.
Employee	View assigned jobs; update progress and internal notes for those jobs.
Administrator	Review all requests; schedule/assign jobs; maintain customers and user roles.

Database and API design

Planned entities: Identity users/roles, Customers, Services, ServiceRequests, Jobs, and JobNotes. Requests have a service and an optional verified customer-account link; a request may produce one job. A job has an assigned employee and many notes. Primary/foreign keys protect relationships. Contact values use text, estimates use decimal currency, and dates/timestamps use date/time types.

Weather integration: Open-Meteo Forecast API, using selected latitude/longitude and daily temperature and precipitation values. Parse JSON on the server; display units, attribution, and retrieval time. The free endpoint is intended for this educational project; review commercial terms before business use.

Design references: Open-Meteo API documentation and terms of use. These are target requirements; load testing, database recovery, authorization testing, and live API testing belong to later implementation milestones.

PART 4 / WIREFRAME 1

Service request

Public screen for visitors and customers. Wireframes show the planned completed workflow.

PAVING SERVICE MANAGEMENT | Home Services Sign in

Request a paving service
Tell us what you need and where the work will take place.

Full name *

[Customer name]

Email address *

[name@example.com]

Phone number *

[Phone number]

Requested service *

[Select service v]

Property address *

[Street, city, state, postal code]

Project details (optional)

[Describe the paving or repair work needed]

Submit service request

Already a customer? Sign in

Interaction: Submit validates input and saves a new request; success shows a reference number. Errors appear beside fields and in a summary. The Part II form only demonstrates validation.

Access: No sign-in is required to submit. Visitors cannot browse stored requests. Signing in permits access only to verified, owned records. Required fields are marked *.

PART 4 / WIREFRAME 2

Staff job dashboard

Administrator view. An employee's view is limited to assigned jobs and permitted updates.

PAVING SERVICE MANAGEMENT | Home Services Sign in

Job dashboard Signed in: Administrator

New	Scheduled	In progress	Completed
4	3	2	8

[Search customer or request] [Status: All v] Filter

ID	Customer	Service	Date	Status	Actions
1001	Sample A	Driveway	[date]	New	View / Assign
1002	Sample B	Repair	[date]	Scheduled	View / Update
1003	Sample C	Sealcoating	[date]	In progress	View / Update

Selected job weather
Location: [city] Forecast: [date] Temperature: [value + units] Rain: [value]
Source: Open-Meteo Retrieved: [time] [Unavailable message when needed]

Interaction: Search/filter narrows records. View opens job details; an administrator assigns work, and the assigned employee updates progress. All data shown here is illustrative.

Access: Server authorization controls every action. Weather supports planning and does not automatically approve or cancel a job. The current shell contains a read-only public preview.

PART 4 / WIREFRAME 3

Sign-in screen

Shared entry point for registered customers, employees, and administrators.

PAVING SERVICE MANAGEMENT | Home Services Sign in

Welcome back
Sign in to manage your paving requests or assigned jobs.

Email address

[name@example.com]

Password

[Password - masked input]

[Generic sign-in error appears here]

Sign in

Create a customer account | Return to services

Interaction: A valid sign-in opens My Requests for a customer, Assigned Jobs for an employee, or the administrator dashboard. Invalid credentials show a generic error.

Access: Roles come from the stored account, not a role selector on this form. New registrations receive only Customer permissions. Sign-in and registration are future features, not present in the shell.